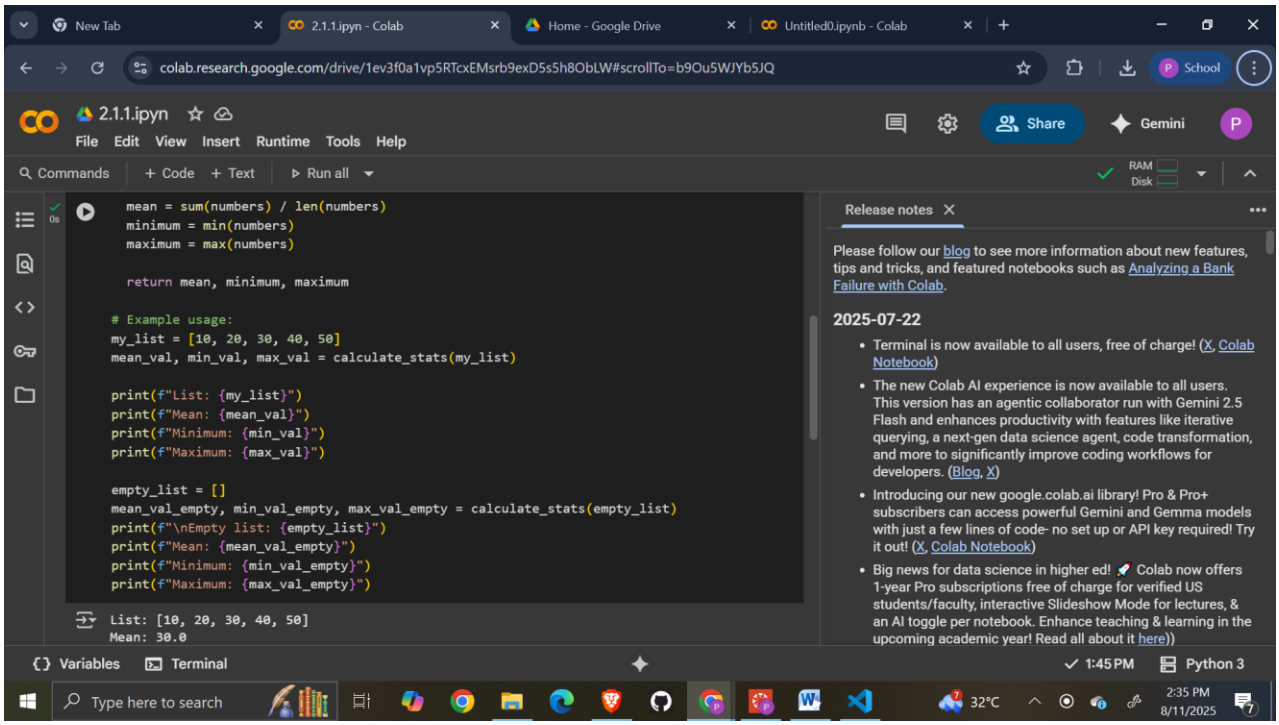
AI Assisted Coding

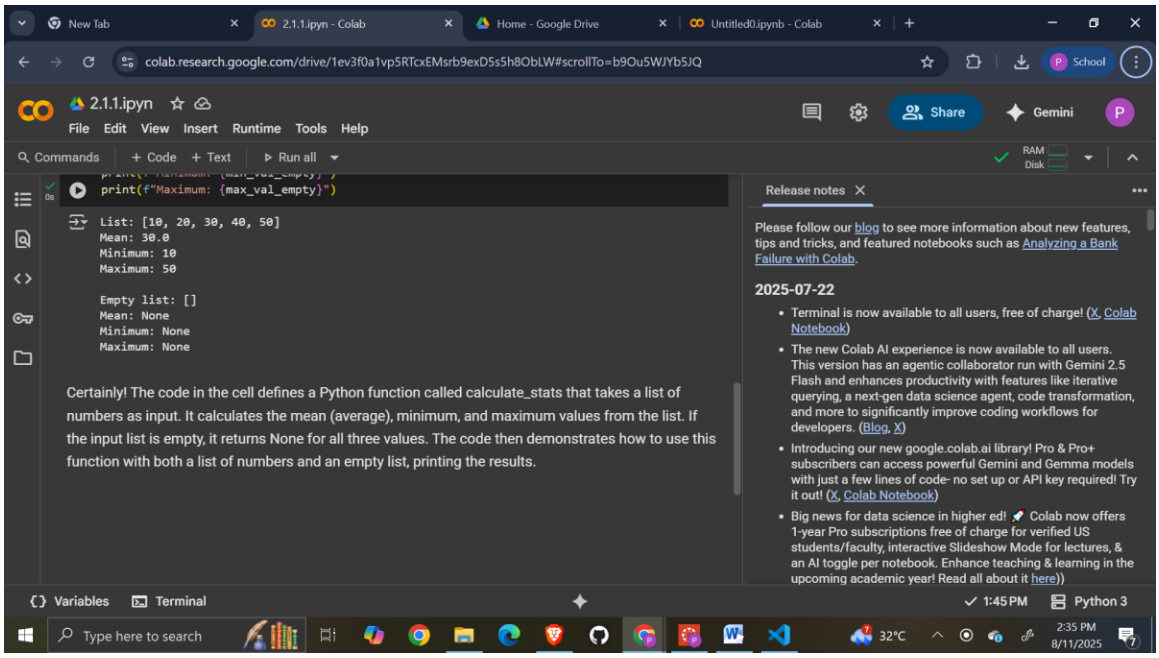
Name: P.VIKAS

Roll Number: 2403A510E6

Batch: 24BTCAICSB05

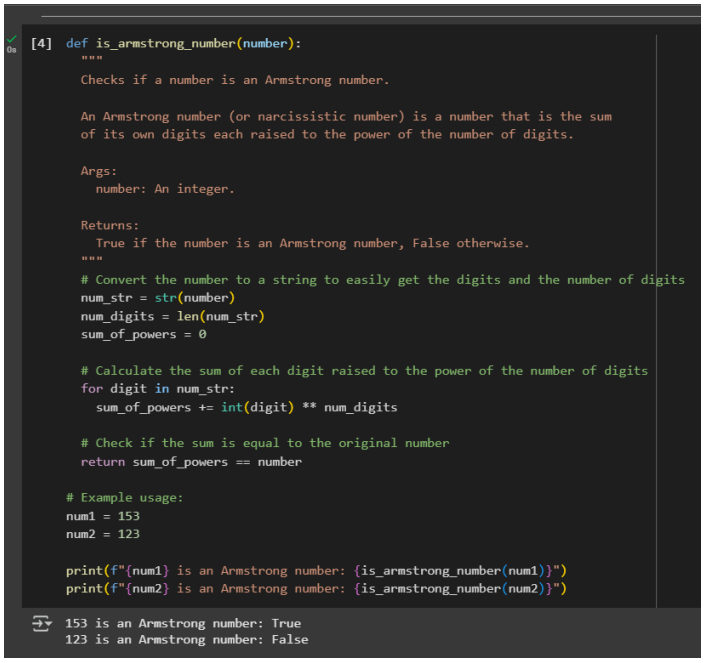
Task Description #1





Task Description #2

GEMINI:



```
[4] def is_armstrong_number(number):
      """
      Checks if a number is an Armstrong number.

      An Armstrong number (or narcissistic number) is a number that is the sum
      of its own digits each raised to the power of the number of digits.

      Args:
        number: An integer.

      Returns:
        True if the number is an Armstrong number, False otherwise.
      """
      # Convert the number to a string to easily get the digits and the number of digits
      num_str = str(number)
      num_digits = len(num_str)
      sum_of_powers = 0

      # Calculate the sum of each digit raised to the power of the number of digits
      for digit in num_str:
        sum_of_powers += int(digit) ** num_digits

      # Check if the sum is equal to the original number
      return sum_of_powers == number

    # Example usage:
    num1 = 153
    num2 = 123

    print(f"{num1} is an Armstrong number: {is_armstrong_number(num1)}")
    print(f"{num2} is an Armstrong number: {is_armstrong_number(num2)}")

153 is an Armstrong number: True
123 is an Armstrong number: False
```

CO-PILOT:

Python Copy

```python
def is_armstrong(number):
    # Convert the number to a string to easily iterate over digits
    digits = str(number)
    num_digits = len(digits)

    # Calculate the sum of each digit raised to the power of num_digits
    total = sum(int(digit) ** num_digits for digit in digits)

    # Check if the total equals the original number
    return total == number

# Example usage
print(is_armstrong(153))  # True
print(is_armstrong(9474)) # True
print(is_armstrong(123))  # False
```

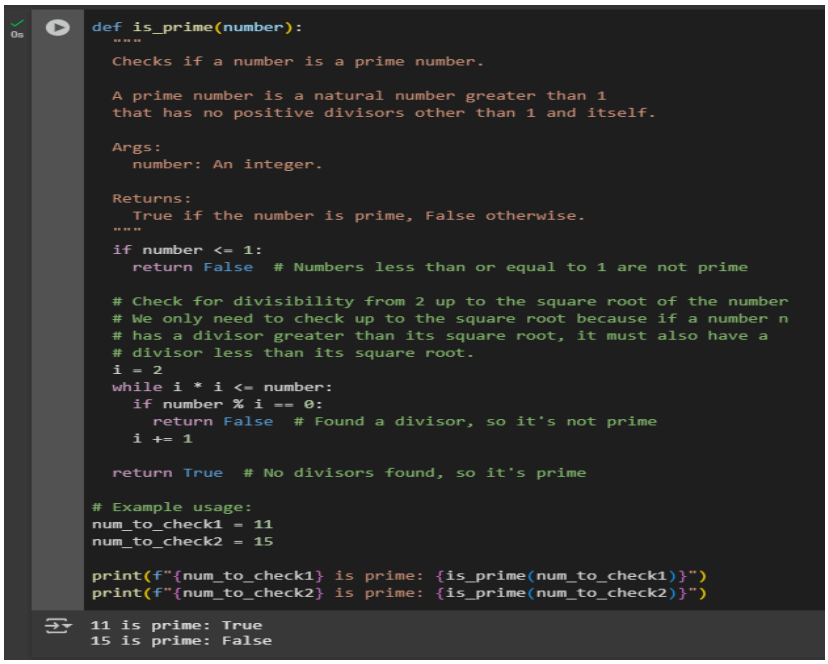
How It Works

- For `153`:
  $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153 \rightarrow$ Armstrong number ✅
- For `9474`:
  $9^4 + 4^4 + 7^4 + 4^4 = 9474 \rightarrow$ Armstrong number ✅
- For `123`:
  $1^3 + 2^3 + 3^3 = 36 \neq 123 \rightarrow$ Not an Armstrong number ❌

COMPARISION:

Feature / Step	Gemini Output (Screenshot 1)	Copilot Output (Screenshot 2)	Observations
Prompt Interpretation	Implements `is_armstrong_number(number)` with detailed docstring explaining Armstrong numbers, arguments, and return value.	Implements `is_armstrong(number)` with concise inline comments, skipping formal docstring.	Gemini focuses on educational clarity with docstrings; Copilot is more concise and direct.
Conversion to String	`num_str = str(number)` and separately counts digits: `num_digits = len(num_str)`.	Similar approach with `digits = str(number)` and `num_digits = len(digits)`.	Both approaches are functionally identical here.
Power Sum Calculation	Uses a `for` loop: `sum_of_powers += int(digit) ** num_digits`.	Uses a generator inside `sum()`: `sum(int(digit) ** num_digits for digit in digits)`.	Copilot uses a more Pythonic, compact one-liner; Gemini uses step-by-step clarity.
Return Condition	`return sum_of_powers == number`	`return total == number`	Identical logic; only variable naming differs.
Example Usage	Prints Armstrong check for 153 (True) and 123 (False).	Prints Armstrong check for 153 (True), 9474 (True), and 123 (False).	Copilot tests an additional well-known Armstrong number (9474).
Extra Explanation	Only outputs print results, no explanation of how it works.	Provides a "How It Works" section breaking down each example step-by-step.	Copilot's explanation makes the concept visually intuitive, with symbols and checks.
Readability	Beginner-friendly with clear steps and docstring.	Compact, elegant, with visual explanation for quick understanding.	Gemini suits detailed learning; Copilot suits quick reference.

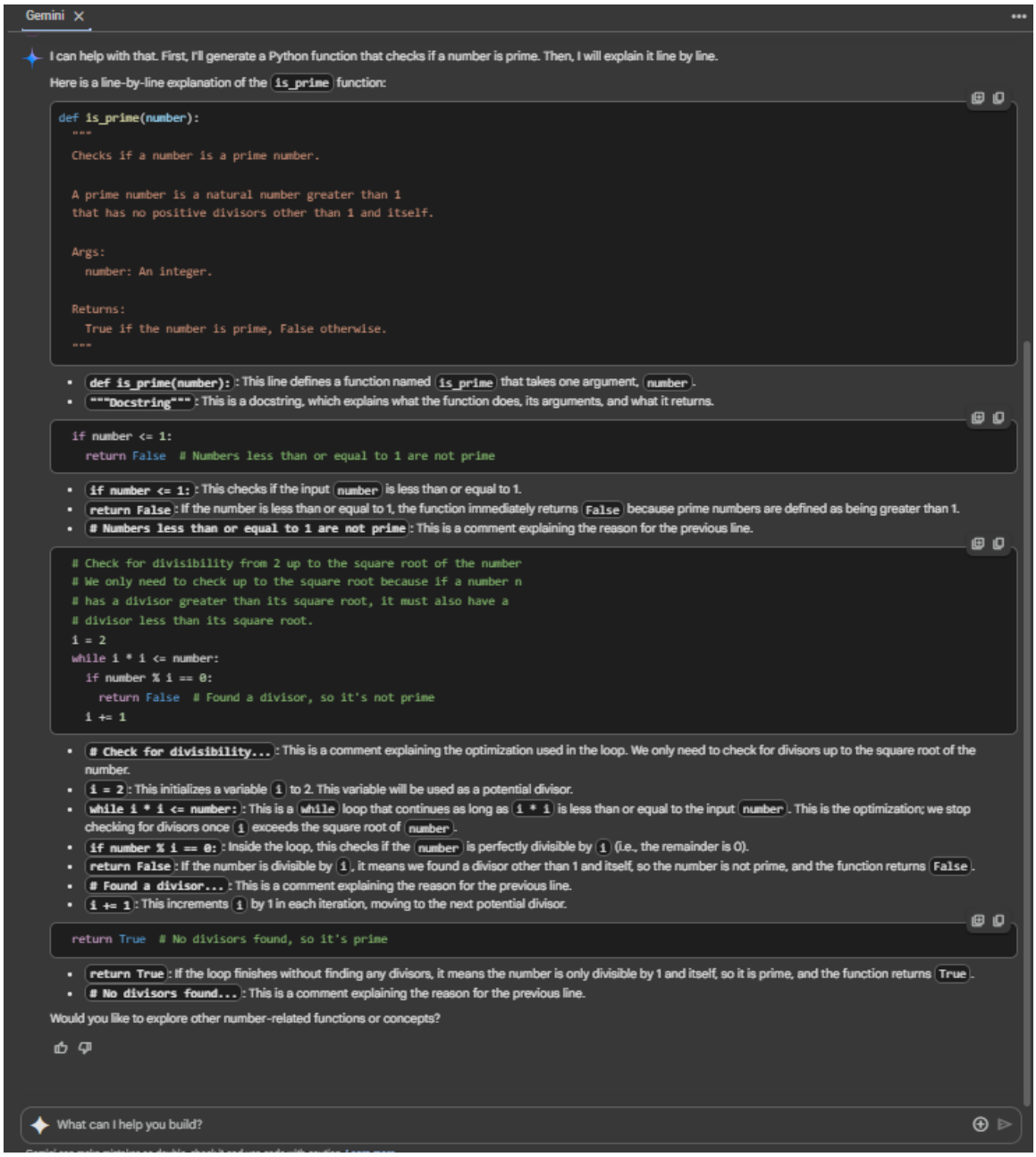
Task Description #3



```python
def is_prime(number):
  """
  Checks if a number is a prime number.

  A prime number is a natural number greater than 1
  that has no positive divisors other than 1 and itself.

  Args:
    number: An integer.

  Returns:
    True if the number is prime, False otherwise.
  """
  if number <= 1:
    return False  # Numbers less than or equal to 1 are not prime

  # Check for divisibility from 2 up to the square root of the number
  # We only need to check up to the square root because if a number n
  # has a divisor greater than its square root, it must also have a
  # divisor less than its square root.
  i = 2
  while i * i <= number:
    if number % i == 0:
      return False  # Found a divisor, so it's not prime
    i += 1

  return True  # No divisors found, so it's prime

# Example usage:
num_to_check1 = 11
num_to_check2 = 15

print(f"{num_to_check1} is prime: {is_prime(num_to_check1)}")
print(f"{num_to_check2} is prime: {is_prime(num_to_check2)}")
```

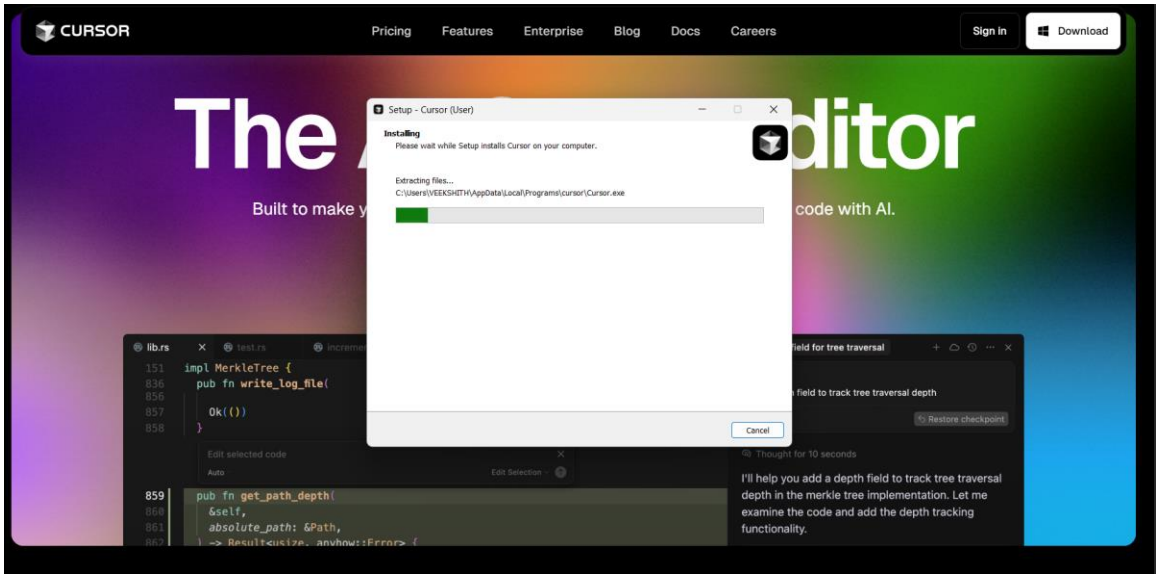
```
11 is prime: True
15 is prime: False
```

LINE BY LINE EXPLANATION:



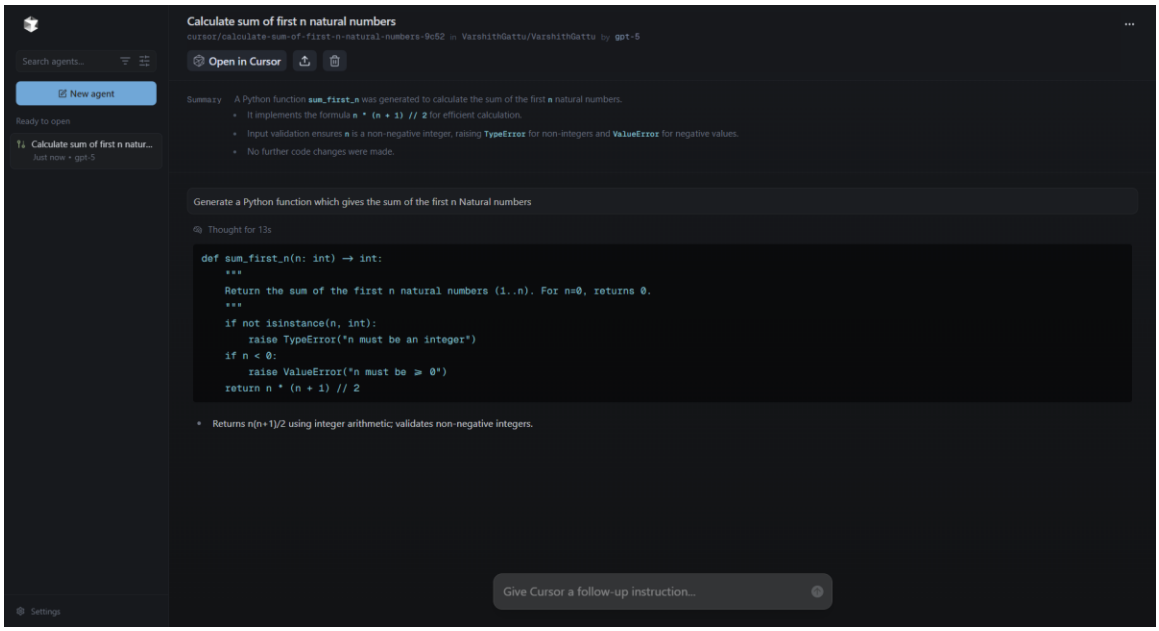
Task Description #4

Cursor AI setup:



Prompts used:

<u>Generate a Python function which gives the sum of the first n Natural numbers.</u>



Task Description #5

Original basic code:

```
numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

even_sum = 0
odd_sum = 0

for num in numbers:
    if num % 2 == 0:
        even_sum += num
    else:
        odd_sum += num

print("Sum of even numbers:", even_sum)
print("Sum of odd numbers:", odd_sum)
```

Sample Output:

```
Sum of even numbers: 30
Sum of odd numbers: 25
```

Refactored code:

```
def sum_even_odd(nums):

    even_sum = sum(n for n in nums if n % 2 == 0)
    odd_sum = sum(n for n in nums if n % 2 != 0)
    return even_sum, odd_sum

numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

even_total, odd_total = sum_even_odd(numbers)

print(f"Sum of even numbers: {even_total}")
print(f"Sum of odd numbers: {odd_total}")
```

Sample Output:

```
Sum of even numbers: 30
Sum of odd numbers: 25
```

Explanation of refactored code:

Line 1: Comment describing what the code does — calculates the sum of odd and even numbers in a tuple.

Line 3: Defines a function sum_even_odd with one parameter nums, which is expected to be a tuple of integers.

Line 4: A **docstring** explaining that the function returns the sum of even and odd numbers from the given tuple.

Line 5: Uses the sum() function with a generator expression to add up all numbers n in nums that are divisible by 2 (n % 2 == 0), meaning they are even.

Line 6: Similarly, sums up all numbers n in nums that are not divisible by 2 (n % 2 != 0), meaning they are odd.

Line 7: Returns both even_sum and odd_sum as a tuple.

Line 10: Creates a sample tuple numbers containing integers from 1 to 10.

Line 13: Calls sum_even_odd(numbers) and stores the two returned values in even_total and odd_total.

Line 16: Prints the sum of even numbers using an f-string for clear formatting.

Line 17: Prints the sum of odd numbers using an f-string.

Output Screenshot: